

```
In [1]: !pip install opencv-python numpy matplotlib
```

```

Collecting opencv-python
  Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_x86_64.whl.metadata (19 kB)
Collecting numpy
  Downloading numpy-2.5.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting matplotlib
  Downloading matplotlib-3.11.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (80 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.63.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (118 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.1 kB)
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/lib/python3.12/site-packages (from matplotlib) (26.0)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in /home/codespace/.local/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/codespace/.local/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_x86_64.whl (73.8 MB)
_____ 73.8/73.8 MB 31.8 MB/s 0:00:026m
0:00:0100:01
Downloading numpy-2.5.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7 MB)
_____ 16.7/16.7 MB 32.9 MB/s 0:00:006m
0:00:01
Downloading matplotlib-3.11.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (10.0 MB)
_____ 10.0/10.0 MB 27.8 MB/s 0:00:006m
0:00:01
Downloading contourpy-1.3.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (362 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.63.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.0 MB)
_____ 5.0/5.0 MB 26.0 MB/s 0:00:00
Downloading kiwisolver-1.5.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
_____ 1.5/1.5 MB 26.4 MB/s 0:00:00
Downloading pillow-12.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9 MB)
_____ 6.9/6.9 MB 31.5 MB/s 0:00:006m0:
00:01

```

Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, numpy, kiwisolver, fonttools, cycler, opencv-python, contourpy, matplotlib
9/9 [matplotlib]9 [matplotlib]on]
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.63.0 kiwisolver-1.5.0 matplotlib-3.11.1 numpy-2.5.2 opencv-python-5.0.0.93 pillow-12.3.0 pyparsing-3.3.2

[notice] A new release of pip is available: 26.0.1 -> 26.2.1

[notice] To update, run: `pip install --upgrade pip`

```
In [3]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV version:", cv2.__version__)
print("NumPy version:", np.__version__)
print("Libraries imported successfully!")
```

OpenCV version: 5.0.0
NumPy version: 2.5.2
Libraries imported successfully!

```
In [4]: import os

print(os.listdir())
```

['exp1.ipynb', 'README.md', 'exp.jpg', '.git']

```
In [5]: image = cv2.imread("exp.jpg")

if image is None:
    print("Image not found!")
else:
    print("Image loaded successfully!")
```

Image loaded successfully!

```
In [6]: image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Original Image



```
In [7]: # Image properties

height, width, channels = image.shape

print("Image Width:", width, "pixels")
print("Image Height:", height, "pixels")
print("Number of Channels:", channels)
print("Image Data Type:", image.dtype)
print("Total Pixels:", width * height)
```

```
Image Width: 290 pixels
Image Height: 387 pixels
Number of Channels: 3
Image Data Type: uint8
Total Pixels: 112230
```

```
In [8]: # Save image in JPEG and PNG formats
```

```
cv2.imwrite("output.jpg", image)
cv2.imwrite("output.png", image)

print("Image saved successfully!")
print("JPEG file: output.jpg")
print("PNG file: output.png")
```

Image saved successfully!

JPEG file: output.jpg

PNG file: output.png

```
In [9]: # Step 5: Color Space Conversion

# Convert BGR image to Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Convert BGR image to HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Convert BGR image to LAB
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

print("Image successfully converted!")
print("Grayscale shape:", gray.shape)
print("HSV shape:", hsv.shape)
print("LAB shape:", lab.shape)
```

Image successfully converted!

Grayscale shape: (387, 290)

HSV shape: (387, 290, 3)

LAB shape: (387, 290, 3)

```
In [10]: # Display different color spaces

plt.figure(figsize=(15, 5))

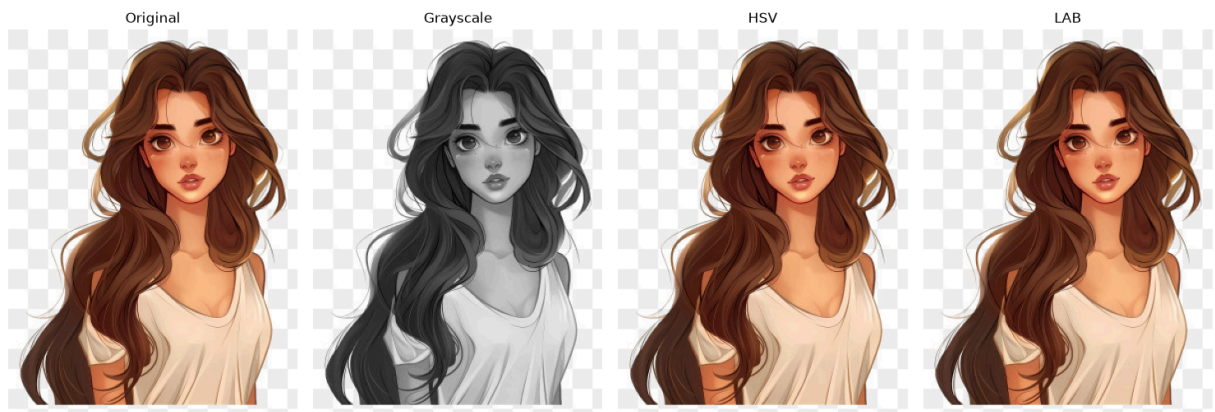
# Original
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

# Grayscale
plt.subplot(1, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# HSV
plt.subplot(1, 4, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

# LAB
plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

plt.tight_layout()
plt.show()
```



```
In [11]: # Step 6.1: Resize Image

resized = cv2.resize(image, (500, 600))

print("Original size:", image.shape)
print("Resized size:", resized.shape)

plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")
plt.show()
```

Original size: (387, 290, 3)

Resized size: (600, 500, 3)

Resized Image



```
In [12]: # Step 6.2: Rotate Image

height, width = image.shape[:2]

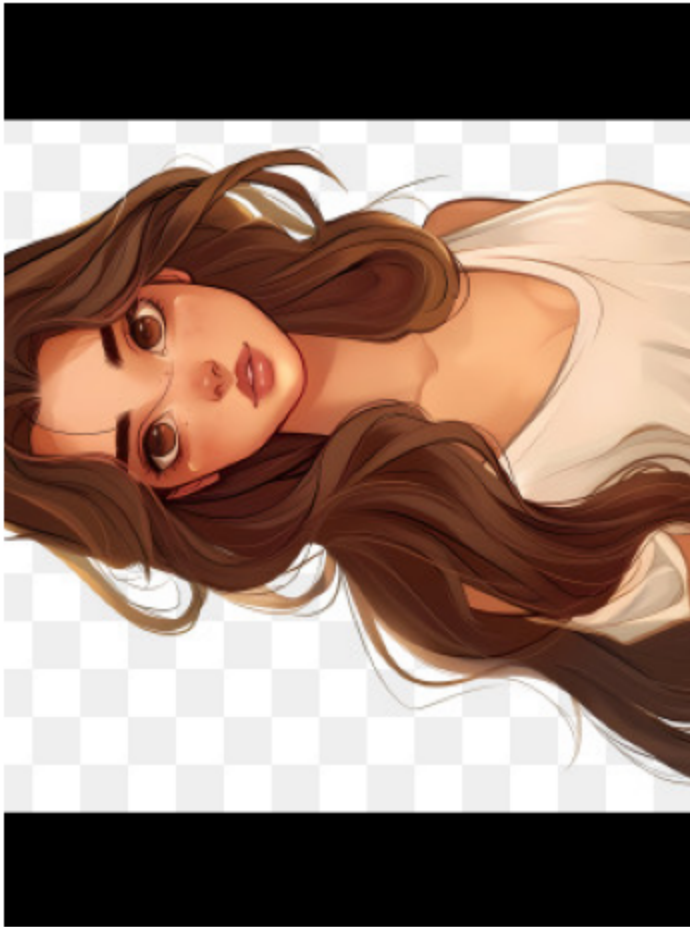
center = (width // 2, height // 2)

rotation_matrix = cv2.getRotationMatrix2D(center, 90, 1)

rotated = cv2.warpAffine(
    image,
    rotation_matrix,
    (width, height)
)

plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image - 90 Degrees")
plt.axis("off")
plt.show()
```

Rotated Image - 90 Degrees



```
In [13]: # Step 6.3: Flip Image

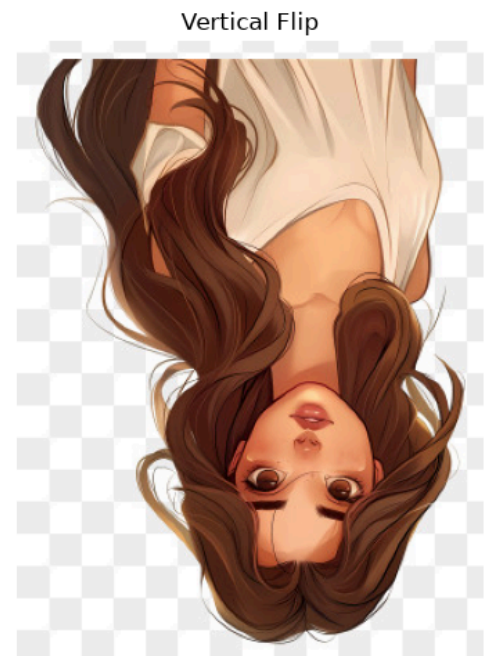
horizontal_flip = cv2.flip(image, 1)
vertical_flip = cv2.flip(image, 0)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
plt.title("Vertical Flip")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Observation

The image was successfully resized, rotated, and flipped. Resizing changes the dimensions of the image, rotation changes its orientation, and flipping creates a mirror-like transformation. These geometric transformations are useful for image preprocessing and data augmentation.

```
In [14]: # Step 7: Image Complement / Negative Image

negative = 255 - image

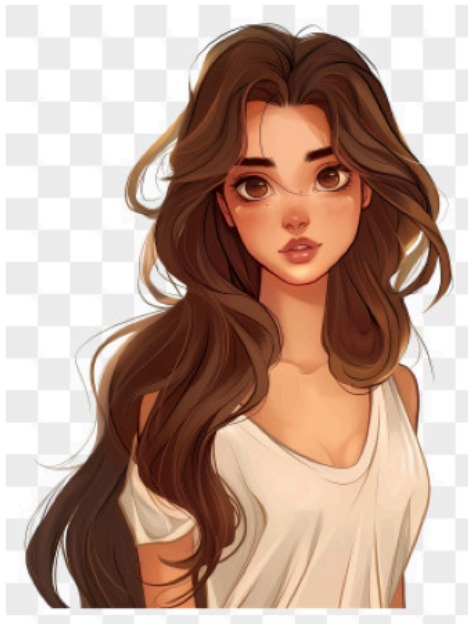
plt.figure(figsize=(12, 5))

# Original Image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

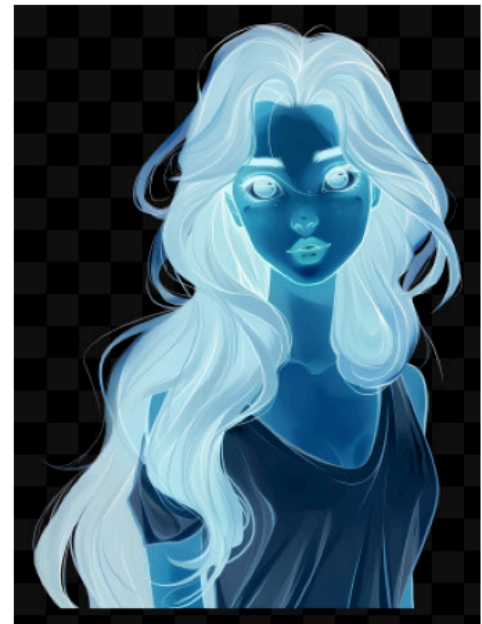
# Negative Image
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Negative Image



Observation

The image complement operation successfully inverted the pixel intensity values. Bright regions became dark and dark regions became bright. Image negative transformation is useful in medical imaging, document analysis, and enhancing certain image details.

```
In [15]: # Step 8: Region of Interest (ROI)

roi = image[50:300, 50:250]

plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()
```

Region of Interest (ROI)



```
In [16]: # ROI Properties

roi_height, roi_width, roi_channels = roi.shape

print("ROI Width:", roi_width)
print("ROI Height:", roi_height)
print("ROI Channels:", roi_channels)
print("ROI Data Type:", roi.dtype)
```

```
ROI Width: 200
ROI Height: 250
ROI Channels: 3
ROI Data Type: uint8
```

Observation

A Region of Interest (ROI) was successfully extracted from the original image. ROI allows a specific portion of an image to be processed separately. It reduces unnecessary computation and is useful for tasks such as object detection, face detection, medical image analysis, and image segmentation.

```
In [17]: # Step 9: Final Comparison of Original and Processed Images

plt.figure(figsize=(15, 12))
```

```
# 1. Original
plt.subplot(3, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# 2. Grayscale
plt.subplot(3, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# 3. HSV
plt.subplot(3, 3, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

# 4. LAB
plt.subplot(3, 3, 4)
plt.imshow(cv2.cvtColor(lab, cv2.COLOR_LAB2RGB))
plt.title("LAB")
plt.axis("off")

# 5. Resized
plt.subplot(3, 3, 5)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")

# 6. Rotated
plt.subplot(3, 3, 6)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

# 7. Horizontal Flip
plt.subplot(3, 3, 7)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

# 8. Negative
plt.subplot(3, 3, 8)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

# 9. ROI
plt.subplot(3, 3, 9)
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("ROI")
plt.axis("off")
```

```
plt.tight_layout()  
plt.show()
```



Final Observation

The fundamental image processing operations were successfully implemented using Python and OpenCV. The image was loaded, displayed, analyzed, saved in different formats, converted into different color spaces, resized, rotated, flipped, complemented, and cropped using ROI.

These preprocessing operations are important because they prepare images for further computer vision tasks such as feature extraction, object detection, image classification, face recognition, medical image analysis, and deep learning.

Conclusion

In this experiment, fundamental image processing techniques were successfully implemented using Python, OpenCV, NumPy, and Matplotlib. The experiment provided practical understanding of image representation, color spaces, geometric transformations, image complement, and Region of Interest extraction. These preprocessing techniques form the foundation for advanced computer vision and deep learning applications.